Guia passo a passo — Hazelcast básico

1. Preparar o ambiente

Java 17 e Python 3 já estavam disponíveis no Ubuntu/WSL usado para desenvolver este projeto. Abra o aplicativo **Ubuntu** no Windows e teste:

```
java -version
javac -version
python3 --version
```

Se os comandos funcionarem, avance ao passo 2. Maven, Docker e conta de nuvem não são necessários.

Se não tiver Ubuntu, abra **PowerShell como administrador** e execute:

```
wsl --install -d Ubuntu
```

Reinicie se solicitado; abra Ubuntu e crie usuário/senha. Depois, **no Ubuntu**:

```
sudo apt update
sudo apt install -y openjdk-17-jdk python3 curl
```

A senha Linux não aparece ao digitar. Essa instalação pode depender de você por exigir senha ou reinicialização. Guia oficial: https://learn.microsoft.com/pt-br/windows/wsl/install

2. Compilar

No Ubuntu:

```
cd /mnt/c/Users/mathe/OneDrive/Documentos/ufop/SD/hazelcast-tp01
bash scripts/build.sh
```

A saída deve terminar com Compilação concluída.. O JAR Hazelcast 5.6.0 acompanha o projeto. O script confere seu SHA-256 e compila as quatro classes Java. Não há instalação separada de um “programa Hazelcast”.

3. Primeira execução: apenas um membro

Abra dois terminais Ubuntu. Em **cada terminal**, entre na pasta com o comando cd do passo 2.

Terminal 1:

```
bash scripts/run.sh membro 5701
```

Terminal 2:

```
bash scripts/run.sh pronto 1
bash scripts/run.sh carregar 10
bash scripts/run.sh somar 10
```

Espere VALIDADO n=10 soma=55 membros=1. Você acabou de usar um mapa distribuído e uma tarefa do Hazelcast, inicialmente com um membro. Nesse estado não há cópia em outro membro; não provoque uma falha esperando recuperar os dados.

4. Transformar em um cluster de três membros

Mantenha os terminais anteriores abertos. Abra mais dois terminais, entre na pasta e execute:

Terminal 3:

```
bash scripts/run.sh membro 5702
```

Terminal 4:

```
bash scripts/run.sh membro 5703
```

No terminal cliente (terminal 2):

```
bash scripts/run.sh pronto 3
```

```
bash scripts/run.sh estado
bash scripts/run.sh somar 10
```

O resultado continua 55. Agora PARCIAIS apresenta uma soma para cada endereço. estado mostra as partições primárias e se as cópias estão sincronizadas. As parcelas variam conforme a distribuição do Hazelcast; o total não deve mudar.

5. Entradas maiores

No terminal cliente:

```
bash scripts/run.sh carregar 1000
bash scripts/run.sh somar 1000
bash scripts/run.sh carregar 10000
bash scripts/run.sh somar 10000
bash scripts/run.sh carregar 100000
bash scripts/run.sh somar 100000
```

Resultados: 500500; 50005000; 5000050000. O parâmetro de somar deve corresponder à entrada carregada. Não carregue outra entrada enquanto uma soma ou acompanhamento estiver executando.

Para medir com aquecimento e três repetições:

```
bash scripts/run.sh benchmark results/minha-medicao.csv
```

O comando mede N=1000, 10000 e 100000 com a quantidade atual de membros. Ele troca o conteúdo do mapa, portanto use fora do acompanhamento. O tempo de carga aparece separado do tempo de soma. O arquivo é acrescentado, não sobrescrito: use nome novo para uma campanha nova.

6. Queda abrupta: versão simples de recuperação

Com três membros e a entrada de 100000 carregada:

```
bash scripts/run.sh pronto 3
bash scripts/run.sh acompanhar 100000
```

O mesmo cliente fará até 60 rodadas, com uma pausa de dois segundos entre elas. Para observar por mais tempo, use acompanhar 100000 120.

Em outro terminal Ubuntu:

```
pgrep -af 'br.edu.ufop.Main membro 5703'
```

Anote o PID da linha do processo Java correspondente e execute (substituindo o texto pelo número):

```
kill -9 PID_DO_MEMBRO_5703
```

Não use um comando que mate todos os processos Java. Observe no cliente:

- Uma rodada pode falhar ou aguardar reorganização.

- Sem reiniciar o cliente, uma rodada seguinte deve mostrar soma=5000050000 membros=2.

- A soma é **refeita por inteiro**. Não existe recuperação sofisticada da tarefa interrompida.

Em outro terminal cliente, você pode conferir:

```
bash scripts/run.sh pronto 2
bash scripts/run.sh estado
```

Não provoque uma segunda falha antes de os backups estarem novamente seguros. As cópias ficam em memória: desligar todo o host perde todos os dados.

7. Aumentar o cluster com o cliente ainda aberto

Enquanto o mesmo acompanhar continua ativo, no terminal que ficou livre:

```
bash scripts/run.sh membro 5704
```

Aguarde o cliente mostrar novamente resultado validado com membros=3. Em outro terminal:

```
bash scripts/run.sh pronto 3
bash scripts/run.sh estado
```

O novo membro recebe partições. A aplicação de acompanhamento permanece aberta, mas pode pausar ou descartar uma rodada durante a mudança. Isso demonstra continuidade do processo cliente, não latência zero nem garantia de completar a mesma tarefa que foi interrompida.

8. Reproduzir todos os testes automaticamente

Encerre os membros manuais com Ctrl+C antes, liberando 5701–5704.

```
python3 scripts/experiments.py
python3 scripts/summarize.py
```

Espere ALL_TESTS_PASSED. Os logs ficam em uma nova pasta de results/; o resumo usa a campanha completa mais recente. A suíte verifica N=0, N=1, N=10, parâmetros inconsistentes, 27 medições, falha e expansão. Encerra somente processos que ela mesma criou.

Esse script é apoio para reproduzir o trabalho: o algoritmo que você precisa explicar está em SomaTask.java e nos métodos carregar/somar do cliente.

9. Slides em LaTeX

O arquivo editável é slides/apresentacao.tex; o PDF pronto é slides/apresentacao.pdf. Não precisa instalar LaTeX para apresentar o PDF.

Para editar sem instalar ferramentas:

1. Acesse https://www.overleaf.com/ e entre em sua conta, se quiser usar o serviço.

2. Crie um projeto em branco.

3. Envie todos os arquivos .tex da pasta slides (apresentacao.tex e resultados.tex).

4. Escolha apresentacao.tex como documento principal.

5. Substitua o campo “Integrantes: preencher nomes e matrículas”.

6. Compile com pdfLaTeX e baixe o PDF.

Alternativa local no Ubuntu, se desejar instalar LaTeX:

```
sudo apt install -y texlive-latex-recommended texlive-latex-extra texlive-lang-portuguese texlive-
fonts-recommended
cd slides
pdflatex -interaction=nonstopmode -halt-on-error apresentacao.tex
pdflatex -interaction=nonstopmode -halt-on-error apresentacao.tex
```

A compilação da entrega utilizou o Tectonic já disponível no plugin; não foi necessário instalar uma distribuição TeX completa. As notas de fala aparecem como \note{...} no fonte e no roteiro de estudo. O PDF padrão mostra somente os slides.

10. Alternativas opcionais

Docker: arquivos preparados, não testados porque Docker não estava disponível. Instale Docker Desktop seguindo https://docs.docker.com/desktop/setup/install/windows-install/ e habilite a integração WSL. Depois:

```
docker compose build
docker compose up -d member1 member2 member3
docker compose run --rm client pronto 3
docker compose run --rm client carregar 100000
docker compose run --rm client acompanhar 100000
```

Em outro terminal, para a falha e expansão:

```
docker compose kill -s SIGKILL member3
```

```
docker compose run --rm client pronto 2
docker compose --profile expansion up -d member4
docker compose run --rm client pronto 3
```

Encerrar: docker compose --profile expansion --profile tools down. Os nomes dos serviços Docker são member1 etc.; o comando Java está em português. Compose aqui usa um host e não equivale a um cluster de máquinas com Kubernetes/Swarm.

Máquinas Linux distintas: copie a pasta para três máquinas na mesma rede privada, instale Java 17 e compile em cada uma. Defina as mesmas seeds e nome de cluster em todos os terminais (substitua os endereços de exemplo):

```
export HZ_CLUSTER=tp01-grupo
export HZ_MEMBERS=10.0.0.11:5701,10.0.0.12:5701,10.0.0.13:5701
```

Em cada membro, defina seu próprio IP em HZ_BIND e inicie membro 5701. Exemplo da primeira máquina:

```
export HZ_BIND=10.0.0.11
bash scripts/run.sh membro 5701
```

No cliente, use as mesmas seeds e os comandos pronto, carregar, somar. Permita TCP 5701 apenas na rede privada entre membros e cliente. Não exponha o laboratório à Internet: não há autenticação/TLS configurados. Infraestrutura remota não foi disponibilizada nem testada nesta entrega.

11. Problemas comuns

- **Comando não encontrado:** use Ubuntu, confira Java/JDK e volte ao passo 1.

- **Porta ocupada:** encerre o membro anterior daquela porta, sem matar outros programas.

- **Connection refused em uma seed:** pode acontecer quando aquela seed ainda não foi iniciada; o cliente deve encontrar um membro vivo.

- **pronto não termina:** verifique o número exato de membros, seeds e nome de cluster.

- **Soma divergente:** use o N carregado, não altere os dados durante o cálculo e espere o cluster estabilizar.

- **AGUARDANDO/RODADA_INTERROMPIDA:** esperado temporariamente durante falhas/migração; deve voltar a validar quando o cluster estiver estável.

- **Memória insuficiente:** reserve cerca de 4 GB livres para vários processos, feche aplicações e evite rodar duas campanhas juntas.